EAD

EDUCAÇÃO A DISTÂNCIA

UNIDADE III

Programação Orientada a Objetos I

Prof. MSc. Tarcísio Peres

Conteúdo da Unidade III

Aprofundamento na linguagem:

- Exceções, delegação.
- Tuplas, coleções.
- Expressões regulares.

- Boas práticas de codificação.
- Princípios SOLID, DRY, KISS, YAGNI e Occam.
- *Design patterns.*

Exceções

- São mecanismos para manipular situações inesperadas ou excepcionais durante a execução de um programa, como tentativas de acessar um índice fora dos limites de uma matriz ou tentativas de dividir por zero.
- Essas anomalias podem originar falhas no código, condições imprevistas, falhas de recursos ou entradas inválidas do usuário. As exceções em C# são representadas por objetos de classes que derivam da classe-base Exception, que é parte do *namespace* System.
- Quando uma condição excepcional é detectada, uma exceção é gerada, ou “lançada”, usando a palavra-chave *throw*, ato que interrompe o fluxo normal de execução do programa e começa a procurar um bloco de código designado para lidar com a exceção, conhecido como *catch.*
  - Blocos *catch* são definidos dentro de construções *try-catch*, em que o bloco *try* contém o código que pode lançar uma exceção, e o bloco *catch* contém o código para tratar a exceção, caso ocorra. Se uma exceção não é capturada por um bloco *catch*, ela propaga-se pela pilha de chamadas, potencialmente alcançando o método Main e, se ainda não for capturada, termina o programa.

Exceções – *throw*

- Lançamento de exceção.

- *Throw.*

```
1.  using System;
2.  class Heroi
3.  {
4.      // Propriedade para armazenar o nome do herói.
5.      public string Nome { get; private set; }
6.      // Construtor da classe que inicializa a propriedade nome.
7.      public Heroi (string nome)
8.      {
9.          if (string.IsNullOrEmpty(nome)) // Verifica se o nome é nulo ou vazio.
10.         {
11.             throw new ArgumentException ("Um herói deve ter um nome!",
nameof (nome)); // Lança uma exceção se o nome é inválido.
12.         }
13.         Nome = nome; // Atribui o nome válido à propriedade Nome.
14.     }
15. }
```

Fonte: autoria própria.

Exceções – *try-catch*

- Bloco *try-catch.*

- É importante utilizar exceções de forma judiciosa, apenas em situações verdadeiramente excepcionais, uma vez que o processo de lidar com exceções pode ser custoso em termos de desempenho.

```
16. class Programa
17. {
18.     static void Main()
19.     {
20.         try
21.         {
22.             // Tentamos criar um herói sem nome, o que é inválido e vai
lançar uma exceção.
23.             Heroi herói = new Heroi ("");
24          }
25.         catch (ArgumentException ex) // Captura qualquer exceção do
tipo ArgumentException.
26.         {
27.             // Escreve a mensagem de erro no console.
28.             Console.WriteLine($"Erro ao criar um herói: {ex.Message}");
29.         }
30.     }
31. }
```

Fonte: autoria própria.

Delegação

- É um recurso que permite encapsular métodos em objetos, sendo crucial para muitos padrões de *design* e práticas de programação e frequentemente usado para implementar eventos e *callbacks*, tornando os programas mais modulares, extensíveis e adaptáveis.
- É implementada por meio do *delegate*, um tipo de referência que pode encapsular um método com um tipo específico de assinatura e retorno.
- Pode referenciar tanto métodos estáticos quanto instância e, uma vez criado, pode ser invocado como qualquer outro método.
- Ao criar um delegate, não se está chamando um método, mas criando uma referência de tipo seguro para esse método.

Delegação – exemplo churrasco

- Vamos criar delegados para as seguintes tarefas: preparar a carne, escolher a música e gelar as bebidas.

- Depois, implementaremos um evento no qual esses delegados serão chamados.

Fonte: autoria própria com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Delegação – exemplo churrasco

- 3 tarefas a serem delegadas aos amigos.
- 3 eventos definidos.
- 3 métodos serão acionados.
  - Operador ?.

```
namespace ChurrascoDelegateExample
{
    // Declarando os tipos de delegado para diferentes tarefas
    delegate void PreparararCarneDelegate(string tipoCarne);
    delegate void EscolherMusicaDelegate(string tipo Musica);
    delegate void Gelar BebidaDelegate(string tipoBebida);

    class Churrasco
    {
        // Eventos baseados nos delegados
        public event PreparararCarneDelegate PreparararCarne;
        public event EscolherMusicaDelegate EscolherMusica;
        public event GelarBebida Delegate GelarBebida;

        // Método para iniciar o churrasco
        public void Iniciar()
        {

            Console.WriteLine("Churrasco começando!");

            // Chama os métodos delegados
            PreparararCarne?. Invoke("Picanha");
            EscolherMusica?. Invoke("Sertanejo");
            GelarBebida?. Invoke("Cerveja");
        }
    }
}
```

Fonte: autoria própria.

Delegação – exemplo churrasco

- Criação do churrasco.
- Atribuição dos métodos.
- Definição dos métodos.

```
class Program
{
        static void Main(string[] args)
        {
                Churrasco churrasco = new Churrasco();
                // Atribuindo métodos aos eventos
                churrasco.PreparareCarne += Churrasqueiro;
                churrasco.EscolherMusica += DJ;
                churrasco.GelarBebida += Bartender;
                // Iniciando o churrasco
                churrasco.Iniciar();
        }
        static void Churrasqueiro (string carne)
        {
                Console.WriteLine($"Preparando a carne: {carne}");
        }
        static void DJ(string musica)
        {
                Console.WriteLine($"Tocando música: {musica}");
        }
        static void Bartender (string bebida)
        {
                Console.WriteLine($"Gelando a bebida: {bebida}");
        }
}
```

Fonte: autoria própria.

Tuplas

- São tipos de dados que podem armazenar múltiplos elementos, possivelmente de tipos diferentes, em uma única unidade.

- Com o lançamento do C# 7.0, o conceito de tuplas foi aprimorado e reformulado para proporcionar uma maneira mais eficiente, sintaticamente agradável e produtiva de representar estruturas de dados leves e imutáveis.

  - As novas tuplas valorizadas são tipos de valor, sendo mais eficientes em termos de desempenho se comparadas com as tuplas de referência anteriores, e podem ser utilizadas de forma integrada, sem necessidade de bibliotecas adicionais.

  - São particularmente úteis quando se deseja retornar mais de um valor de um método sem a necessidade de criar um tipo de classe ou estrutura adicional, o que facilita a escrita de códigos mais concisos e expressivos.

Tuplas

- Tupla zeus.

- Tupla nomeada hades.

```
1.  using System;
2.
3.  class Programa
4.  {
5.  static void Main()
6.  {
7.          // Cria uma tupla representando um deus grego, contendo nome,
domínio e símbolo.
8.          var zeus = ("Zeus", "Céu e Trovão", "Águia");
9.
10.         // Exibe as informações da tupla na console.
11.         Console.WriteLine($"{zeus. Item1} é o deus do {zeus. Item2) e seu
símbolo é a (zeus. Item3)."");
12.
13.         // Cria uma tupla nomeada, proporcionando maior clareza ao código.
14.         var hades = (Nome: "Hades", Dominio: "Submundo", Simbolo:
"Capacete da Invisibilidade");
15.
16.         // Exibe as informações da tupla nomeada.
17.         Console.WriteLine($" (hades. Nome) é o deus do (hades. Dominio}
e seu símbolo é o {hades. Simbolo}.");
18.         }
19  }
```

Fonte: autoria própria.

Interatividade

Qual das seguintes afirmações é correta sobre o tratamento de exceções em C#?

a) Todas as exceções devem ser tratadas com blocos *try* e *catch*, e o uso de *finally* é opcional e raramente usado.

b) As exceções em C# são usadas para controlar o fluxo normal do programa.

c) Uma exceção em C# é um evento que ocorre durante a execução de um programa e que interrompe o fluxo normal das instruções do programa, sendo capturada e tratada.

d) Não é possível criar exceções personalizadas.

e) O tratamento de exceções em C# é considerado uma prática de programação de baixo desempenho e deve ser evitado.

Resposta

Qual das seguintes afirmações é correta sobre o tratamento de exceções em C#?

a) Todas as exceções devem ser tratadas com blocos *try* e *catch*, e o uso de *finally* é opcional e raramente usado.

b) As exceções em C# são usadas para controlar o fluxo normal do programa.

c) Uma exceção em C# é um evento que ocorre durante a execução de um programa e que interrompe o fluxo normal das instruções do programa, sendo capturada e tratada.

d) Não é possível criar exceções personalizadas.

e) O tratamento de exceções em C# é considerado uma prática de programação de baixo desempenho e deve ser evitado.

Coleção

- Refere-se a uma classe ou estrutura que permite agrupar múltiplos elementos, os quais podem ser variáveis, instâncias de objetos ou até outras coleções, dependendo do tipo de coleção utilizado.
- Coleções são vitais porque permitem que os programadores manipulem conjuntos de dados de forma mais eficiente e organizada, facilitando a implementação de diversas operações, como ordenação, filtragem, busca, entre outras.
- O C# oferece várias classes e interfaces de coleções na BCL, agrupadas principalmente nos *namespaces* System.Collections, System.Collections.Generic, System.Collections.Concurrent e System.Collections.Immutable. A classe-base de coleções no *namespace* System.Collections oferece tipos como ArrayList e HashTable.
    - O *namespace* System.Collections.Generic inclui coleções genéricas, como List e Dictionary, mais modernas e seguras em termos de tipo, permitindo definir o tipo de dados que a coleção pode armazenar e oferecendo maior segurança e *performance* ao evitar conversões de tipo desnecessárias.

Coleção

- Quando for necessário acessar coleções a partir de múltiplas *threads* simultaneamente, o namespace System.Collections.Concurrent oferece coleções como ConcurrentDictionary, otimizadas para garantir a consistência dos dados em ambientes *multithreaded.*
- A coleção BitArray armazena *bits* (0s e 1s) de maneira eficiente em termos de memória, sendo útil quando é necessário representar e manipular conjuntos de valores booleanos.
- A LinkedList é uma coleção que implementa uma lista duplamente encadeada, sendo particularmente útil quando são necessárias inserções e deleções frequentes de elementos.
- A coleção SortedSet, por sua vez, representa uma coleção de elementos não duplicados, armazenados em ordem, útil quando é essencial manter os elementos ordenados.
    - O *namespace* System.Collections.Immutable oferece coleções que, uma vez criadas, não podem ser modificadas, e qualquer operação que altere a coleção retornará uma nova instância com as alterações, mantendo a original inalterada.
    - Coleções podem expandir e contrair dinamicamente à medida que os dados são adicionados ou removidos.

Expressões regulares

- Proporcionam uma maneira sofisticada e eficiente de processar texto, permitindo executar operações complexas de manipulação de *string.*
- Fundamentam-se na teoria das linguagens formais e consistem em sequências de caracteres que formam um padrão, servindo para descrever um conjunto de *strings*. Com esse padrão, é possível identificar, combinar, substituir ou dividir *strings* de forma precisa e flexível, utilizando regras específicas e estruturadas.
- Oriundas do *namespace* System.Text.RegularExpressions, as expressões regulares em C# são úteis quando se lida com validação de dados, pesquisa e análise de texto, em que padrões como os de *e-mails*, números de telefone e códigos postais precisam ser identificados dentro de um texto maior, como documentos, *logs* ou entradas de usuário.
    - Ao se trabalhar com expressões regulares, emprega-se a classe Regex, que oferece uma série de métodos para trabalhar com padrões. Esses métodos permitem executar diferentes tipos de operação, como pesquisa e correspondência, substituição de texto e divisão de *strings*, utilizando o padrão definido pela expressão regular.

Expressões regulares – exemplos práticos

- ^\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*$ é uma regex que valida a maioria dos endereços de *e-mails* comuns. Ela começa com um ou mais caracteres alfanuméricos (\w+), seguidos por zero ou mais ocorrências de caracteres permitidos em *e-mails* ([-+.']\w+), um @, seguido pelo domínio que tem uma estrutura semelhante, e finaliza com o TLD (*top-level domain*) que pode ter partes separadas por pontos.
- <([A-Za-z][A-Za-z0-9]*)\b[^>]*>(.*?)</\1> é a regex utilizada para encontrar *tags* HTML e seu conteúdo. Ela captura o nome da tag ([A-Za-z][A-Za-z0-9]*), ignora os atributos dentro da *tag* de abertura (\b[^>]*), captura o conteúdo ((.*?)) e depois encontra a *tag* de fechamento correspondente (</\1>). É importante notar que regex não é a ferramenta ideal para parsear HTML complexo, mas pode ser útil para casos simples.
- ^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[A-Za-z\d]{8,}$ é uma regex que assegura que a *string* contém pelo menos oito caracteres, pelo menos uma letra maiúscula, uma minúscula e um número. É útil para validar senhas que precisam cumprir critérios específicos de segurança.

Expressões regulares

- É crucial compreender a sintaxe específica, que pode envolver uma variedade de caracteres especiais, quantificadores, agrupamentos e âncoras, cada um com significados e propósitos diferentes.

Tipo	Descrição
Ponto (.)	Corresponde a qualquer caractere único, com exceção de uma nova linha. É frequentemente utilizado quando o caractere exato é desconhecido ou variável.
Asterisco (*)	Busca por zero ou mais ocorrências do caractere ou grupo anterior. Por exemplo, no padrão a*, qualquer quantidade de caracteres a será uma correspondência válida, inclusive uma *string* vazia.
Mais (+)	Corresponde a uma ou mais ocorrências do caractere ou grupo anterior. No padrão a+, por exemplo, uma ou mais ocorrências de a são necessárias para uma correspondência.
Interrogação (?)	Busca por zero ou uma ocorrência do caractere ou grupo anterior, tornando-o opcional na busca de correspondências.
Chaves ({n}, {n,}, {n,m})	Especificam a quantidade exata, mínima ou intervalo de ocorrências do caractere ou grupo anterior. Por exemplo, a{3} busca três ocorrências consecutivas de a, enquanto a{3,} busca três ou mais, e a{3,5) busca entre três e cinco inclusivamente.
Caret (^)	Símbolo de âncora que representa o início de uma linha ou *string*, utilizado para encontrar correspondências no início de uma linha.
Cifrão ($)	É outro símbolo de âncora, porém representa o final de uma linha ou *string*. É utilizado para encontrar correspondências ao final de uma linha.
Colchetes ([ ])	Definem uma classe de caracteres e correspondem a qualquer caractere único dentro dos colchetes. Por exemplo, [ab] corresponde a a ou b.
Parênteses (( ))	Agrupam parte da expressão regular, permitindo aplicar quantificadores a esse grupo de caracteres. Por exemplo, (ab)+ corresponde a uma ou mais ocorrências de ab.

Fonte: adaptado de: livro-texto.

Expressões regulares

- Classe Regex.
- Método IsMatch.

```
15.     // Expressão regular para encontrar entidades cujos nomes começam com 'A'
16.     string padrao = "A";
17.
18.     // Regex compilada utilizando o padrão definido
19.     Regex regex = new Regex (padrao);
20.
21.     // Lista para armazenar as entidades que correspondem ao padrão
22.     List<string> entidades Correspondentes = new List<string>();

23.
24. // Varre a lista de entidades mitológicas e verifica quais correspondem ao padrão da expressão
    regular

25.     foreach (string entidade in entidadesMitologicas)
26.     {

27.             if (regex.IsMatch(entidade))
28.                     entidadesCorrespondentes.Add(entidade);
29.     }

30.
31. // Exibe as entidades que correspondem ao padrão
32.     Console.WriteLine("Entidades que correspondem ao padrão:");
33.     foreach (string entidade in entidadesCorrespondentes)
34.             Console.WriteLine(entidade);
35.     }
36. }
```

Fonte: autoria própria.

Melhorando a programação

- Melhorar a programação em C# envolve uma combinação de práticas recomendadas e uso efetivo de ferramentas e recursos associados.
- O primeiro passo para isso é sem dúvida uma compreensão profunda da sintaxe e das funcionalidades básicas da linguagem. Como em qualquer disciplina, os fundamentos são cruciais: dominar conceitos fundamentais – como classes, herança, polimorfismo, interfaces e delegação – fornece uma base sólida para construir habilidades mais avançadas.
- A familiaridade com a sintaxe básica e avançada melhora a leitura e a escrita de códigos.
- À medida que a Microsoft continua a desenvolver e evoluir o C#, novas versões da linguagem são lançadas, trazendo novos recursos e funcionalidades.
    - É essencial os programadores se manterem atualizados com essas mudanças, pois estar ciente das atualizações mais recentes e saber como e quando usá-las pode proporcionar melhorias significativas na eficiência e na qualidade do trabalho.
    - A prática deliberada é uma das maneiras mais eficazes de internalizar o conhecimento e transformá-lo em habilidade.

Melhorando a programação

- Melhorar a programação significa não apenas codificar regularmente, mas também desafiar-se com projetos e problemas novos e variados.
- Ao enfrentar e superar desafios, os programadores não apenas aplicam o que sabem, mas também identificam áreas onde podem faltar conhecimento ou compreensão.
- A qualidade do código é tão importante quanto sua funcionalidade.
- Adotar práticas recomendadas de programação e *design* de *software* pode garantir que o código seja modular, reutilizável e fácil de manter.
- Ações como escrever testes unitários e adotar uma abordagem de desenvolvimento orientado a testes também podem elevar a qualidade e a robustez do código.
    - O ecossistema ao redor do C# e .NET é vasto. Utilizar eficazmente as ferramentas disponíveis, como o Visual Studio, pode acelerar o desenvolvimento e tornar o processo mais suave.
    - Ferramentas de análise estática de código, por exemplo, podem identificar problemas de código antes que se tornem problemas mais graves em produção.

Melhorando a programação

- Uma das melhores maneiras de melhorar em qualquer campo é aprender com os outros. Participar de comunidades (como o Stack Overflow), conferências de desenvolvimento e grupos de usuários locais pode trazer novas perspectivas e *insights*, pois o *feedback* é inestimável, e os pares podem oferecer soluções para problemas que talvez nunca tenhamos considerado.
- Aprimorar a programação em C# exige uma combinação de compreender a linguagem em profundidade, manter-se atualizado com as mudanças, praticar regularmente, adotar medidas recomendadas, dominar ferramentas e aprender com a comunidade.
    - Como qualquer habilidade, <u>leva tempo e esforço</u>, mas, com <u>dedicação</u>, a maestria é alcançável.

Fonte: autoria própria com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Boas práticas de codificação

- Um dos primeiros aspectos a considerar quando se fala em boas práticas de codificação é clareza. A clareza do código é diretamente proporcional à facilidade de sua manutenção e *debugging.*
- Isso é possível adotando nomes de variáveis, classes e métodos descritivos. Em vez de usar nomes de variáveis (como x ou y), é muito mais proveitoso usar nomes como velocidade ou distância – nomes que não só ajudam o desenvolvedor original, mas também qualquer outro que possa trabalhar no código no futuro.
- Outra prática essencial é utilizar comentários de maneira eficaz. Em vez de comentar cada linha, o que pode tornar o código poluído, é preferível comentar blocos de código ou funções complexas que possam não ser imediatamente óbvias para quem lê.
    - Isso serve como bússola para guiar desenvolvedores pela complexidade inerente ao *software.*
    - XML comments, uma característica particular do C#, permitem que desenvolvedores documentem diretamente no código, o que facilita a geração automática de documentação.

Boas práticas de codificação

- Comentários XML não são como os tradicionais, que são ignorados pelo compilador; em vez disso, são projetados para ser processados por ferramentas que podem gerar documentação externa ou fornecer dicas de ferramentas no ambiente de desenvolvimento.

- Um comentário XML começa com /// (três barras) em vez do usual // (duas barras) para comentários de linha única; isso sinaliza para o compilador que o comentário deve ser tratado de maneira especial.

```
/// <summary>
/// Classe que representa operações mitológicas.
/// </summary>
public class OperacoesMitologicas
{
    /// <summary>
    /// Calcula a soma de dois poderes mitológicos.
    /// </summary>
    /// <param name="poderZeus">Poder de Zeus.</param>
    ///  <param name="poderAthena">Poder de Athena.</param>
    /// <returns>0 total combinado de poder.</returns>
    public int SomaPoderes (int poderZeus, int poderAthena)
    {
        return poderZeus + poderAthena;
    }
```

Fonte: autoria própria.

XML Comments – *tags*

- <summary> oferece uma breve descrição de método, classe, propriedade etc., e é comum ver essa *tag* logo acima das definições de método ou classe.
- <param> descreve um parâmetro de método ou função, com um atributo “name” que especifica qual parâmetro está sendo descrito.
- <returns> descreve o valor retornado por um método ou função.
- Estas são apenas algumas das *tags* mais comuns, mas existem muitas outras, incluindo <remarks>, <example> e <exception>, que permitem aos desenvolvedores fornecer uma documentação mais detalhada.
  - Ao usar ferramentas como Doxygen ou Sandcastle, os desenvolvedores podem gerar documentações HTML ou outros formatos a partir desses comentários XML.
  - Além disso, ambientes de desenvolvimento, como o Visual Studio, usam esses comentários para fornecer dicas de ferramentas durante a codificação, fazendo os desenvolvedores entenderem melhor o propósito e o uso de métodos, classes ou propriedades específicas.

Práticas desejáveis

- Evitar a duplicação desnecessária de código também é importante, o que pode ser feito usando métodos, classes-base e interfaces para reutilizar o código de maneira eficiente.
- Além de evitar a duplicação, a modularização do código é extremamente útil. Dividir o código em métodos e classes menores, cada um com sua responsabilidade, permite que os desenvolvedores compreendam, testem e mantenham o código com muito mais facilidade.
- A reutilização de código é incentivada em C# por meio de herança e composição. No entanto, deve-se preferir a composição à herança, uma vez que esta pode levar a hierarquias complexas e frágeis, enquanto aquela oferece maior flexibilidade, permitindo que os objetos sejam construídos a partir de pequenos componentes que podem ser facilmente reutilizados e testados isoladamente.
    - Ao evitar a repetição, reduzimos o potencial de erros, já que qualquer mudança necessária no futuro só precisará ser feita em um local.

Práticas desejáveis

- Tipagem forte é uma das características mais poderosas do C#. Aproveitando-se disso, deve-se sempre preferir tipos explícitos em vez de var, exceto quando o tipo é óbvio a partir do contexto imediato, pois isso torna o código mais legível e evita possíveis confusões sobre os tipos de variáveis usadas.
- A estilização do código também desempenha papel fundamental nas boas práticas. Tarefas simples – como manter uma consistência na indentação, usar chaves para delimitar blocos de código e manter uma estrutura consistente em todo o código – podem fazer uma grande diferença na legibilidade.
- O uso adequado de convenções de nomenclatura também facilita o entendimento e a manutenção do código.
- O desenvolvimento orientado a testes e os testes unitários, muitas vezes com o uso de *frameworks* como NUnit ou xUnit, garante que o código funcione como esperado e que regressões sejam identificadas rapidamente.

Interatividade

Qual das afirmações é correta sobre o uso de expressões regulares e coleções em C#?

a) É impossível usar expressões regulares com coleções em C#.

b) A classe Regex pode ser usada para analisar *strings* e armazenar resultados correspondentes em coleções, como List<string>.

c) A combinação de expressões regulares e coleções em C# é limitada ao uso com Dictionary<TKey, TValue>, em que TKey é um padrão de expressão regular.

d) Expressões regulares em C# não podem retornar coleções de correspondências.

e) Você só pode usar a classe Regex do *namespace* System.Text.RegularExpressions para encontrar correspondências em uma *string.*

Resposta

Qual das afirmações é correta sobre o uso de expressões regulares e coleções em C#?

a) É impossível usar expressões regulares com coleções em C#.

b) A classe Regex pode ser usada para analisar *strings* e armazenar resultados correspondentes em coleções, como List<string>.

c) A combinação de expressões regulares e coleções em C# é limitada ao uso com Dictionary<TKey, TValue>, em que TKey é um padrão de expressão regular.

d) Expressões regulares em C# não podem retornar coleções de correspondências.

e) Você só pode usar a classe Regex do *namespace* System.Text.RegularExpressions para encontrar correspondências em uma *string.*

Princípio SOLID

- Representa uma série de cinco princípios fundamentais do *design* orientado a objetos e programação. Promovido por Robert C. Martin, destina-se a melhorar a qualidade e manutenibilidade do código-fonte.
- O princípio da responsabilidade única, representado pela letra S, enfatiza que uma classe deve ter apenas uma razão para mudar; no ambiente C#, isso pode ser traduzido em manter nossas classes e métodos concisos, possivelmente usando classes parciais ou organizando-as em *namespaces* específicos para garantir clareza e coesão.
- Por exemplo, uma classe que lida com operações de banco de dados em C# não deve também ser responsável por lógica de interface do usuário.
- O princípio aberto-fechado, simbolizado pela letra O, pode ser evidenciado pelo poderoso sistema de interfaces e herança da linguagem. Em vez de modificar uma classe existente, os desenvolvedores são encorajados a estender suas funcionalidades por meio de herança ou implementação de interfaces, garantindo que o código-base existente permaneça intacto, reduzindo a possibilidade de erro.

Princípio SOLID

- A letra L refere-se ao princípio da substituição de Liskov, que se manifesta ao garantir às subclasses que estendem ou implementam superclasses e interfaces respeitar as expectativas originais. Uma subclasse deve ser intercambiável com sua superclasse sem quebrar a aplicação.
- Já o princípio da segregação da interface, representado pela letra I, ganha relevância ao considerarmos a modularidade e flexibilidade que C# oferece. Em vez de sobrecarregar uma classe ou interface com múltiplos métodos, os desenvolvedores podem criar múltiplas interfaces, cada uma focada em uma funcionalidade específica, tornando o código mais legível e mantendo classes e interfaces enxutas e focadas.
- A letra D simboliza o princípio da inversão de dependência, que advoga pela dependência em abstrações e não em implementações concretas, sendo vivamente expresso em C# por meio de injeção de dependências. Com *frameworks* como o ASP.NET Core, os desenvolvedores podem facilmente inverter dependências, tornando o código mais modular e testável.

Princípio SOLID – analogia: construção de uma casa

- ***S**ingle Responsibility*: cada trabalhador tem uma especialização – um eletricista cuida da fiação, um encanador dos canos e assim por diante. Cada um tem responsabilidade única.
- ***O**pen/Closed*: a estrutura da casa (*design*, *layout*) está definida e fechada para alterações, mas você pode adicionar elementos, como pintura ou decoração, sem alterar a estrutura.
- ***L**iskov Substitution*: se um carpinteiro for substituído por outro, o novo deve ser capaz de realizar as mesmas tarefas sem alterar o resultado do trabalho.
- ***I**nterface Segregation*: os trabalhadores não precisam saber como realizar todos os trabalhos da casa (por exemplo, um pintor não precisa saber sobre instalação elétrica).
  Eles só precisam conhecer e utilizar as ferramentas e técnicas do seu ofício.
- ***D**ependency Inversion*: a construção não depende diretamente de uma marca específica de ferramentas ou materiais. Em vez disso, depende de abstrações como "ferramentas de corte" ou "materiais isolantes", permitindo flexibilidade na escolha de fornecedores e ferramentas.

Princípio DRY

- Significa *don't repeat yourself* (não se repita, em tradução livre), é uma das pedras angulares da programação moderna e advoga pela redução da repetição de informações ou lógica em sistemas de *software*, incentivando desenvolvedores a reutilizar o código e evitar a duplicação desnecessária.
- A linguagem C# fornece uma série de mecanismos – como herança, interfaces, delegados e métodos de extensão – que podem evitar a repetição de código.
- Quando uma lógica é repetida em várias partes de um programa, não só aumenta a probabilidade de erro – pois qualquer mudança na lógica exige modificações em múltiplos locais –, mas também dificulta o entendimento e a manutenção do código.
    - Exemplo: um código que processa informações de clientes. Se em várias partes do código houver trechos que formatam a apresentação do nome do cliente de maneira específica e essa lógica precisar ser alterada, o desenvolvedor terá que lembrar de atualizar essa lógica em todos os lugares. Em contrapartida, se o código seguisse o princípio DRY, essa lógica estaria centralizada em um único método ou classe.

Princípio DRY – analogia: receita de bolo

- DRY (*Don't Repeat Yourself*): pense em usar uma receita de bolo. Em vez de escrever a mesma receita várias vezes para diferentes bolos, você mantém uma cópia da receita original e a refere sempre que precisar, garantindo consistência e evitando erros.

Fonte: autoria própria com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Princípio KISS

- Significa *keep it simple, stupid* (“deixe simples, idiota”, em tradução livre), é uma abordagem filosófica em engenharia e *design* que prioriza a simplicidade.
- Convida-nos a abordar problemas e soluções com clareza e direção, evitando uma complexidade desnecessária que pode se tornar um fardo ao longo do tempo.
- Em C#, uma linguagem rica e poderosa, pode ser tentador mergulhar nas profundezas de seus recursos avançados, no entanto, o KISS nos lembra que a solução mais simples é frequentemente a melhor. Isso não significa evitar a inovação ou as características avançadas da linguagem; significa, ao contrário, empregar essas ferramentas de forma criteriosa e apenas quando elas oferecem valor real.
    - Por exemplo, é possível criar estruturas complexas de herança, usar reflexão extensivamente ou empregar padrões de *design* avançados, mas, se uma funcionalidade pode ser alcançada mais diretamente sem essas ferramentas, optar pela simplicidade geralmente resulta em código mais legível, mantível e eficiente.

Princípio KISS

- Outro exemplo: se um simples *loop foreach* serve ao propósito, talvez não seja necessário introduzir uma expressão LINQ complicada.

- Código simples frequentemente se traduz em menos erros, pois há menos locais onde as coisas podem dar errado – o que também torna o processo de depuração mais gerenciável, além de o código simples ser frequentemente mais rápido, pois evita operações desnecessárias ou convoluções.

  - O maior benefício em adotar o KISS, especialmente num ambiente de desenvolvimento colaborativo, é a facilidade de comunicação. Quando um código é revisitado – seja por um colega ou pelo próprio desenvolvedor – a simplicidade facilita o entendimento e a extensão do trabalho, tornando todo o ciclo de vida do *software* mais eficaz.

Princípio KISS – analogia: *kit* de montagem

- KISS (*Keep It Simple, Stupid*): construir um móvel usando um *kit* simples com instruções claras e ferramentas básicas, em vez de projetar um do zero com ferramentas e técnicas complexas.
  A simplicidade torna o processo mais eficiente e menos propenso a erros.

Fonte: autoria própria com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Princípio YAGNI

- Acrônimo de *you aren't gonna need it* ("você não vai precisar disso", em tradução livre) – é um princípio de desenvolvimento de *software* que aconselha programadores a não adicionar funcionalidades até que sejam realmente necessárias.
- Essa filosofia visa combater a tendência de sobre-engenharia, ou seja, a inclinação para criar sistemas excessivamente complexos baseados em necessidades percebidas do futuro.
- A implementação do princípio YAGNI no contexto do desenvolvimento em C# revela a importância de equilibrar as capacidades expansivas da linguagem com a disciplina de desenvolvimento enxuto.
    - Por exemplo, se um desenvolvedor estiver criando uma aplicação em C# e acreditar que ela pode precisar de múltiplos métodos de autenticação, ele pode ser induzido a implementar vários sistemas de *login* desde o início. No entanto, se a demanda atual requer apenas uma solução simples, o princípio YAGNI sugere começar apenas com essa solução. Quando surgirem requisitos adicionais, o código pode ser adaptado e expandido conforme necessário.

Princípio YAGNI

- Adotar YAGNI significa resistir à tentação de usar todas as características avançadas e *frameworks* disponíveis, a menos que haja necessidade clara e presente. Isso não apenas mantém o código mais limpo e compreensível, mas também reduz o tempo de desenvolvimento e teste, pois desenvolvedores não trabalham em recursos desnecessários.
- Não promove negligência nem falta de planejamento; em vez disso enfatiza a importância da adaptabilidade.
- Em vez de tentar prever e preparar-se para todos os cenários futuros possíveis em C#, é mais prático e eficiente construir soluções para os desafios atuais e adaptar-se à medida que novos requisitos emergem.
- Em síntese, YAGNI é uma lembrança crucial para os desenvolvedores, para manter o foco no presente, construindo soluções que atendam às necessidades atuais de forma eficiente e evitando a complexidade desnecessária que pode vir da sobre-engenharia.

Princípio YAGNI – analogia: mala de viagem

- YAGNI (*You Aren't Gonna Need It*): ao arrumar uma mala de viagem, ao invés de levar itens “só por precaução” que provavelmente não usará, leve apenas o essencial. Isso torna a mala mais leve e a viagem mais simples.

Fonte: autoria própria com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Princípio de Occam

- A navalha de Occam é uma heurística filosófica que sugere que, quando confrontados com várias explicações ou soluções concorrentes para um problema, deve-se escolher a mais simples, que requeira o menor número de pressupostos.
- Ao programar em C#, assim como em qualquer linguagem de programação, é comum sermos apresentados a diversos caminhos possíveis para resolver um problema ou implementar uma funcionalidade. Com o vasto conjunto de ferramentas, bibliotecas e padrões à disposição, pode ser tentador criar soluções complexas ou utilizar abordagens sofisticadas para tarefas que poderiam ser abordadas de forma mais simples.
- Incorporar o princípio de Occam no desenvolvimento em C# é lembrar-se de que a solução mais direta e descomplicada é frequentemente a mais apropriada.
    - Por exemplo, se um desenvolvedor estiver considerando um padrão de *design* complexo ou uma biblioteca de terceiros para uma tarefa em C#, o princípio de Occam sugere pausar e considerar se uma abordagem mais simples – talvez usando recursos nativos da linguagem ou ferramentas já existentes no projeto – poderia ser igualmente eficaz.

Princípio de Occam – analogia: pense em cavalos

- *Occam's Razor* (Navalha de Occam): se você ouve cascos, pense em cavalos, não em zebras (a menos que esteja na África). Em outras palavras, a explicação mais simples (cavalos) é geralmente a correta, ao invés de algo mais complexo e improvável (zebras).

Fonte: autoria própria com tecnologia DALL-E, uma ferramenta de inteligência artificial desenvolvida pela OpenAI.

Nuances – KISS, YAGNI e Occam

- KISS, YAGNI e navalha de Occam, à primeira vista, podem parecer conceitos redundantes, pois todos enfatizam a importância da simplicidade e advertem contra a complexidade desnecessária.

- Ao examinar suas origens e nuances, percebemos que, embora se sobreponham em muitas áreas, cada um oferece uma perspectiva única e aborda diferentes aspectos do desenvolvimento e tomada de decisão.

  - KISS busca simplicidade de *design.*
  - YAGNI evita o excesso no desenvolvimento.
  - A navalha de Occam escolhe uma entre múltiplas soluções.
  - No cerne de todos os princípios, há um chamado à clareza, eficiência e intencionalidade na criação de *software*.

Interatividade

Qual das afirmações é **<u>incorreta</u>**?

a) SOLID: conjunto de cinco princípios para *design* orientado a objetos, visando tornar o *software* mais compreensível, flexível e sustentável.

b) DRY: evitar duplicações no código e a reutilização, para facilitar a manutenção e reduzir a possibilidade de inconsistências.

c) KISS: sistemas de forma simples e descomplicada, evitando complexidade desnecessária.

d) YAGNI: não adicionar funcionalidades até que elas sejam realmente necessárias.

e) *Occam's Razor*: na solução de problemas, prefira as explicações mais simples e com menos suposições, evitando a complexidade desnecessária.

Resposta

Qual das afirmações é **<u>incorreta</u>**?

a) SOLID: conjunto de cinco princípios para *design* orientado a objetos, visando tornar o *software* mais compreensível, flexível e sustentável.

b) DRY: evitar duplicações no código e a reutilização, para facilitar a manutenção e reduzir a possibilidade de inconsistências.

c) KISS: sistemas de forma simples e descomplicada, evitando complexidade desnecessária.

d) YAGNI: não adicionar funcionalidades até que elas sejam realmente necessárias.

e) *Occam's Razor*: na solução de problemas, prefira as explicações mais simples e com menos suposições, evitando a complexidade desnecessária.

Design patterns

- Na engenharia de *software*, padrões de *design* têm papel crucial na construção de sistemas robustos, escaláveis e mantidos com facilidade.

- Essas soluções reutilizáveis, surgidas da necessidade de resolver problemas recorrentes na fase de *design* e implementação de sistemas, transcendem o simples ato de codificar; elas representam um profundo entendimento de como os sistemas funcionam e como componentes distintos interagem entre si.

  - Embora não seja presa a uma linguagem específica, cada linguagem, com suas peculiaridades, pode influenciar como determinado padrão é abordado ou implementado. Em C#, a riqueza das bibliotecas-padrão e as características da linguagem tornam a implementação desses padrões particularmente intuitiva.

Design patterns – categorias

- Padrões criacionais: se concentram em como os objetos são criados e instanciados. Esses padrões reduzem as dependências sobre classes concretas e são fundamentais para o desacoplamento de código.

- Padrões estruturais: lidam com a composição de classes ou objetos para formar estruturas maiores. Eles ajudam a garantir que, quando uma parte da estrutura muda, o sistema inteiro não precisa mudar.

- Padrões comportamentais: são usados principalmente para gerenciar, coordenar e comunicar entre objetos. Esses padrões ajudam a definir como os objetos interagem e distribuem responsabilidades.

Design patterns – categorias

Criacional

- Abstract Factory
- Builder
- Factory Method
- Prototype
- Singleton

Estrutural

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Comportamental

- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor

Design patterns – criacionais

- Padrões criacionais.

Singleton	▪ Garante que uma classe tenha apenas uma instância e fornece um ponto de acesso global a ela.
Factory Method	▪ Define uma interface para criar um objeto, mas permite que as subclasses decidam qual classe instanciar.
Abstract Factory	▪ Fornece uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas.
Builder	▪ Separa a construção de um objeto complexo de sua representação, de forma que o mesmo processo possa criar diferentes representações.
Prototype	▪ Especifica os tipos de objeto a criar usando uma instância prototípica e cria objetos copiando esse protótipo.

Design patterns – criacionais: exemplo de aplicação

- Uma empresa precisa gerar uma variedade de relatórios financeiros, como balanços patrimoniais, demonstrações de resultados e análises de fluxo de caixa.
- Tais relatórios são essenciais para a tomada de decisões, acompanhamento do desempenho e planejamento estratégico.
- Cada relatório pode ter diferentes formatos e requerer diferentes conjuntos de dados.
- O padrão Factory é ideal para esse cenário. Ele permite a criação de objetos (neste caso, diferentes tipos de relatórios financeiros) sem especificar a classe exata que será criada.

```
public abstract class RelatorioFinanceiro
{
    public abstract void GerarRelatorio();
}
public class BalancoPatrimonial : RelatorioFinanceiro
{
    public override void GerarRelatorio()
    {
        // Lógica para gerar o balanço patrimonial
    }
}
public class DemonstracaoResultados : RelatorioFinanceiro
{
    public override void GerarRelatorio()
    {
        // Lógica para gerar a demonstração de resultados
    }
}
```

Fonte: autoria própria.

Design patterns – criacionais: exemplo de aplicação

- Gerar Relatório de Balanço Patrimonial: RelatorioFinanceiro balancoPatrimonial = fabricaRelatorioFinanceiro.CriarRelatorio ("BalancoPatrimonial");

- Gerar Demonstração de Resultados: RelatorioFinanceiro demonstracaoResultados = fabricaRelatorioFinanceiro.CriarRelatorio ("DemonstracaoResultados");

```
public class FabricaRelatorioFinanceiro
{
    public RelatorioFinanceiro CriarRelatorio (string tipoRelatorio)
        {
            switch (tipoRelatorio)
            {
                case "BalancoPatrimonial":
                    return new BalancoPatrimonial();
                case "Demonstracao Resultados":
                    return new DemonstracaoResultados();
                // Pode adicionar mais tipos de relatório conforme necessário
                default:
                    throw new ArgumentException("Tipo de relatório inválido");
            }
    }
}
```

Fonte: autoria própria.

Design patterns – estruturais

- Padrões estruturais.
- Há outros.

Adapter

- Permite que classes com interfaces incompatíveis trabalhem juntas. Envolve uma classe existente com uma nova interface.

Bridge

- Desacopla uma abstração de sua implementação para que as duas possam variar independentemente.

Composite

- Compõe objetos em estruturas de árvore para representar hierarquias de parte-todo. Permite que clientes tratem objetos individuais e composições de objetos uniformemente.

Facade

- Fornece uma interface unificada para um conjunto de interfaces em um subsistema. Define uma interface de nível mais alto que facilita o uso do subsistema.

Proxy

- Fornece um substituto ou espaço reservado para outro objeto para controlar o acesso a ele.

Design patterns – estruturais: exemplo de aplicação

- Uma empresa que utiliza um sistema de gestão financeira interno e decide integrar um novo provedor de serviços de pagamento externo.
- O novo provedor tem uma interface de API diferente da que o sistema interno espera.
- Podemos criar um adaptador (Adapter) para traduzir as chamadas da API do sistema interno para o formato esperado pelo novo provedor de pagamento.

```
// Interface esperada pelo sistema interno
public interface IServicoPagamento
{
    void ProcessarPagamento (string detalhes Pagamento);
}
// Implementação da API do novo provedor de pagamento
public class ProvedorPagamento Externo
{
    public void ExecutarTransacaoPagamento (string detalhes Transacao)
    {
        // Lógica para processar o pagamento com o provedor externo
    }
}
```

Fonte: autoria própria.

Design patterns – estruturais: exemplo de aplicação

- ProvedorPagamentoExterno provedorExterno = new ProvedorPagamentoExterno();
- IServicoPagamento servicoPagamento = new AdaptadorPagamento(provedorExterno);
- servicoPagamento.ProcessarPagamento("Detalhes do Pagamento");

```
// Adapter que traduz a chamada do sistema interno para o provedor externo
public class Adaptador Pagamento : IServicoPagamento
{
    private readonly ProvedorPagamentoExterno _provedorPagamentoExterno;

    public AdaptadorPagamento (ProvedorPagamentoExterno provedorPagamentoExterno)
    {
        _provedor PagamentoExterno = provedorPagamentoExterno;
    }

    public void ProcessarPagamento (string detalhesPagamento)
    {
        // Traduz os detalhes do pagamento para o formato esperado pelo provedor extern
        string detalhesTransacao = TraduzirDetalhes (detalhesPagamento);
        _provedorPagamento Externo. ExecutarTransacaoPagamento(detalhesTransacao);
    }
    private string TraduzirDetalhes (string detalhes)
    {
        // Lógica de tradução
        return "Detalhes traduzidos";
    }
}
```

Fonte: autoria própria.

Design patterns – comportamentais

- Padrões comportamentais.
- Há outros.

Command

- Encapsula uma solicitação como um objeto, permitindo que usuários parametrizem clientes com diferentes solicitações, enfileirem solicitações e suportem operações reversíveis.

Iterator

- Fornece uma maneira de acessar elementos de uma coleção de objetos sequencialmente sem expor sua representação subjacente.

Observer

- Define uma dependência um-para-muitos entre objetos, de modo que, quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

State

- Permite que um objeto altere seu comportamento quando seu estado interno muda. O objeto parecerá ter mudado de classe.

Strategy

- Define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. A estratégia permite que o algoritmo varie, independentemente dos clientes que o utiliza.

Design patterns – comportamentais: exemplo de aplicação

- Uma empresa de comércio eletrônico que precisa manter seus clientes atualizados sobre o *status* dos pedidos.
- Sempre que o *status* de um pedido muda (por exemplo, de "processando" para "enviado"), o sistema deve notificar automaticamente o cliente.
- Cenário ideal para o padrão Observer.

```
1.  // Interface Observer
2.  public interface IClienteObserver
3.  {
4.      void Atualizar(Pedido pedido);
5.  }
```

Fonte: autoria própria.

```
// Interface Subject
public class Pedido
{
    public string Status { get; private set; }
    private List<IClienteObserver> _observers = new List<IClienteObserver>();
    public void AdicionarObserver(IClienteObserver observer)
    {
        _observers.Add(observer);
    }
    public void RemoverObserver(IClienteObserver observer)
    {
        _observers. Remove (observer);
    }
    public void AlterarStatus (string novoStatus)
    {
        Status = novoStatus;
        NotificarObservers();
    }
    private void NotificarObservers()
    {
        foreach (var observer in_observers)
        {
            observer.Atualizar(this);
        }
    }
}
```

Design patterns – comportamentais: exemplo de aplicação

```
// Implementação concreta do Observer
public class Cliente: IClienteObserver
{
    public void Atualizar(Pedido pedido)
    {
        // Enviar notificação ao cliente sobre a mudança de status
        Console.WriteLine($"Notificação ao cliente: O status do seu pedido é agora '{pedido. Status}'.");
    }
}
```

Fonte: autoria própria.

- Pedido pedido = new Pedido();
- Cliente cliente = new Cliente();

- pedido.AdicionarObserver(cliente);
- pedido.AlterarStatus("enviado");

Design patterns – criacionais: analogia restaurante

- Padrões criacionais.

Singleton	▪ O restaurante tem uma central única de atendimento ao cliente para todas as filiais. Não importa em qual filial o cliente esteja, ele sempre entrará em contato com essa central única para *feedback*, perguntas ou reclamações.
Factory Method	▪ Cada estação de preparo no restaurante (como a estação de hambúrgueres, bebidas etc.) sabe como preparar um item específico do menu, mas a estação exata que prepara o item pode variar dependendo do pedido do cliente.
Abstract Factory	▪ O menu oferece diferentes "*kits*" de refeição, como um combo de hambúrguer, batatas fritas e refrigerante, que combinam bem juntos, mas sem especificar a receita ou marca exatas de cada item.
Builder	▪ O cliente pode personalizar seu combo. O processo de escolher o tipo de hambúrguer, adicionar ou remover ingredientes, escolher o tamanho das batatas fritas e a bebida permite ao cliente construir seu pedido passo a passo, resultando em uma refeição personalizada.
Prototype	▪ Se um cliente frequente sempre pede o mesmo combo personalizado, o restaurante pode "lembrar" seu pedido como um protótipo. Da próxima vez que o cliente visitar, ele pode simplesmente pedir "o usual“.

Design patterns – estruturais: analogia restaurante

- Padrões estruturais.

Adapter

- O restaurante expandiu internacionalmente e agora recebe pedidos em vários idiomas, sem mudar o sistema interno de processamento de pedidos.

Bridge

- O restaurante oferece um menu que varia de acordo com a região (por exemplo, opções vegetarianas, itens locais específicos), sem alterar a estrutura básica de como os menus são criados e apresentados.

Composite

- Um cliente pode fazer um pedido que é uma combinação de vários itens menores (como um combo de hambúrguer, batatas fritas e refrigerante), facilitando o gerenciamento e a entrega dos itens como um pacote unificado.

Facade

- Os clientes não precisam saber sobre os detalhes da preparação da comida; eles simplesmente fazem seus pedidos no balcão, que fornece uma interface simplificada para o processo mais complexo de preparação na cozinha.

Proxy

- O restaurante tem um serviço de entrega *on-line* em que os clientes fazem pedidos por um aplicativo. Ele atua como um intermediário entre pedidos dos clientes e os encaminha para a cozinha do restaurante.

Design patterns – comportamentais: analogia restaurante

- Padrões comportamentais.

Command

- Cada pedido feito pelos clientes é como um "comando" que contém todas as informações necessárias para preparar e servir a comida. Esse comando é então passado para a cozinha e eventualmente para o serviço de entrega ou balcão.

Iterator

- Considere como os clientes percorrem o menu para escolher os itens. Eles podem começar dos aperitivos, passar para os pratos principais e depois para as sobremesas.

Observer

- Quando um pedido é feito no balcão, os cozinheiros são notificados instantaneamente. Além disso, os clientes recebem uma notificação quando seu pedido está pronto.

State

- O *status* de cada pedido no restaurante pode mudar ao longo do tempo - de "recebido" para "em preparação", "pronto para entrega" e "entregue“.

Strategy

- Diferentes métodos de preparo de alimentos podem ser considerados como diferentes "estratégias". Por exemplo, fritar, assar ou grelhar são diferentes estratégias de cozimento.

Interatividade

Qual padrão define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis, permitindo que o algoritmo varie independentemente dos clientes que o utilizam?

a) Singleton.
b) Bridge.
c) Proxy.
d) Strategy.
e) Observer.

Resposta

Qual padrão define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis, permitindo que o algoritmo varie independentemente dos clientes que o utilizam?

a) Singleton.
b) Bridge.
c) Proxy.
d) Strategy.
e) Observer.

ATÉ A PRÓXIMA!